

SYED SUALEH ABBAS 10303

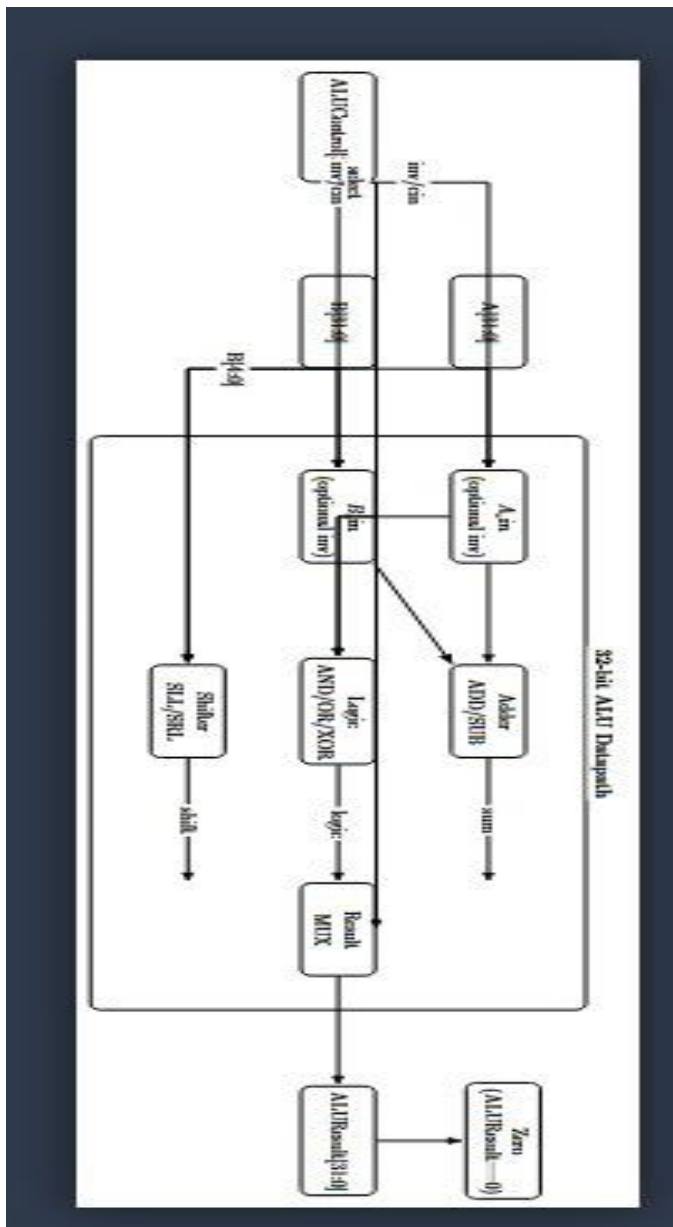
SYED MOHSIN NAQVI

SYED ZAID NAQVI

Lab 6: Design and FPGA Implementation of the ALU for a Single-Cycle RISC-V Processor

Task 1: ALU Block Diagram Design

Block diagram of the ALU.





Task 2: Verilog Description and Testbench Development

Verilog source file for ALU

```
`timescale 1ns / 1ps
/////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 02/17/2026
// Design Name: 32-bit ALU
// Module Name: ALU32bit
// Description: Self-contained 32-bit ALU with multiple operations
/////////////////////////////////////////////////////////////////

module ALU32bit(
    input [31:0] A,      // First operand
    input [31:0] B,      // Second operand
    input [3:0] ALUControl, // Control signals to select operation
    output reg [31:0] ALUResult, // Result of ALU
    output Zero          // Zero flag
);

// Internal wires
wire [31:0] A_in, B_in;
wire [31:0] and_out, or_out, xor_out, nor_out, sll_out, srl_out, sum, slt_out;
wire [31:0] carry;

// Invert inputs if needed (for SUB or NOR)
assign A_in = (ALUControl[3]) ? ~A : A;
assign B_in = (ALUControl[2]) ? ~B : B;

// Logic operations
assign and_out = A_in & B_in;
assign or_out = A_in | B_in;
assign xor_out = A_in ^ B_in;
assign nor_out = ~(A_in | B_in);

// Shift operations
assign sll_out = A << B[4:0]; // Logical left shift
assign srl_out = A >> B[4:0]; // Logical right shift

// Ripple carry adder for ADD/SUB
assign sum[0] = A_in[0] ^ B_in[0] ^ ALUControl[2]; // cin = 1 for subtraction
assign carry[0] = (A_in[0] & B_in[0]) | (B_in[0] & ALUControl[2]) | (A_in[0] &
```

```
ALUControl[2]);
```

```
genvar i;
generate
for (i = 1; i < 32; i = i + 1) begin : adder
    assign sum[i] = A_in[i] ^ B_in[i] ^ carry[i-1];
    assign carry[i] = (A_in[i] & B_in[i]) | (B_in[i] & carry[i-1]) | (A_in[i] & carry[i-1]);
end
endgenerate
```

```
// SLT (Set on Less) - only LSB = 1 if A < B
assign slt_out = 32'b0 | (sum[31] ? 32'b1 : 32'b0);
```

```
// Output multiplexer based on ALUControl
always @(*) begin
    case(ALUControl)
        4'b0000: ALUResult = and_out; // AND
        4'b0001: ALUResult = or_out;  // OR
        4'b0010: ALUResult = sum;     // ADD
        4'b0110: ALUResult = sum;     // SUB
        4'b0011: ALUResult = xor_out; // XOR
        4'b1100: ALUResult = nor_out; // NOR
        4'b0111: ALUResult = slt_out; // SLT
        4'b0100: ALUResult = sll_out; // SLL
        4'b0101: ALUResult = srl_out; // SRL
        default: ALUResult = 32'b0;
    endcase
end
```

```
// Zero flag
assign Zero = (ALUResult == 32'b0);
```

```
endmodule
```



Testbench file

```
`timescale 1ns / 1ps
/////////////////////////////////////////////////////////////////
// Testbench for ALU32bit
/////////////////////////////////////////////////////////////////

module ALU32bit_tb;

// Inputs
reg [31:0] A;
reg [31:0] B;
reg [3:0] ALUControl;

// Outputs
wire [31:0] ALUResult;
wire Zero;

// Instantiate the ALU
ALU32bit uut (
    .A(A),
    .B(B),
    .ALUControl(ALUControl),
    .ALUResult(ALUResult),
    .Zero(Zero)
);

initial begin
    // Initialize inputs
    A = 32'd15;
    B = 32'd10;

    // Test AND
    ALUControl = 4'b0000; // AND
    #10;

    // Test OR
    ALUControl = 4'b0001; // OR
    #10;

    // Test ADD
    ALUControl = 4'b0010; // ADD
    #10;
```



```
// Test SUB
ALUControl = 4'b0110; // SUB
#10;

// Test XOR
ALUControl = 4'b0011; // XOR
#10;

// Test NOR
ALUControl = 4'b1100; // NOR
#10;

// Test SLT (A < B)
A = 32'd5;
B = 32'd10;
ALUControl = 4'b0111; // SLT
#10;

// Test SLT (A > B)
A = 32'd20;
B = 32'd10;
ALUControl = 4'b0111; // SLT
#10;

// Test SLL (shift left)
A = 32'h0000_000F; // 15
B = 32'd2; // shift by 2
ALUControl = 4'b0100; // SLL
#10;

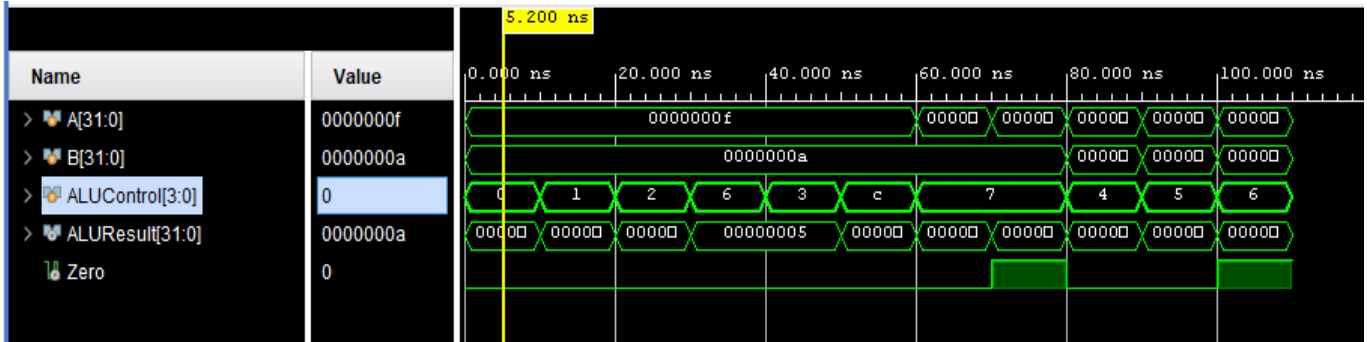
// Test SRL (shift right)
A = 32'h0000_00F0; // 240
B = 32'd4; // shift by 4
ALUControl = 4'b0101; // SRL
#10;
// Test Zero flag
A = 32'd5;
B = 32'd5;
ALUControl = 4'b0110; // SUB → 0
#10;

$finish;
end

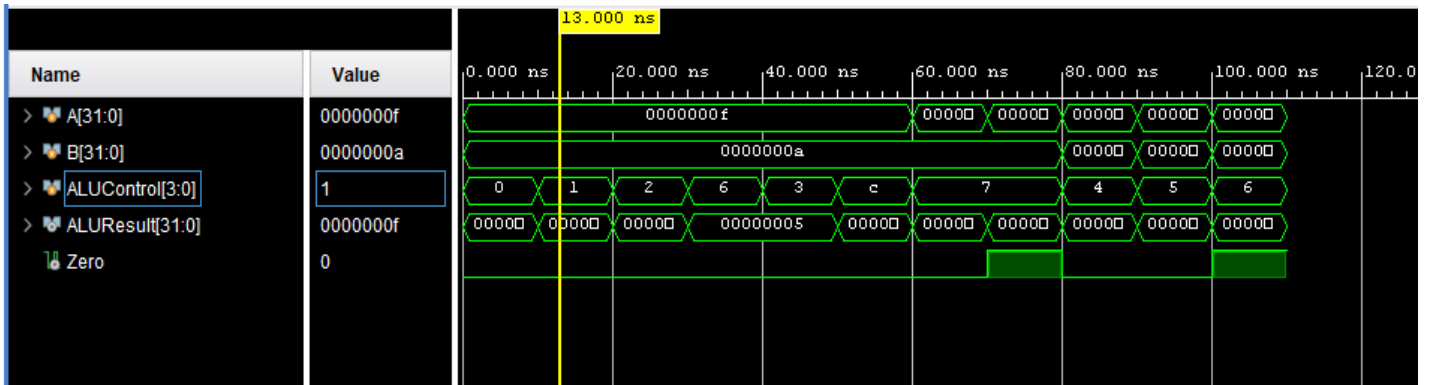
endmodule
```

Simulation waveforms showing correct operation

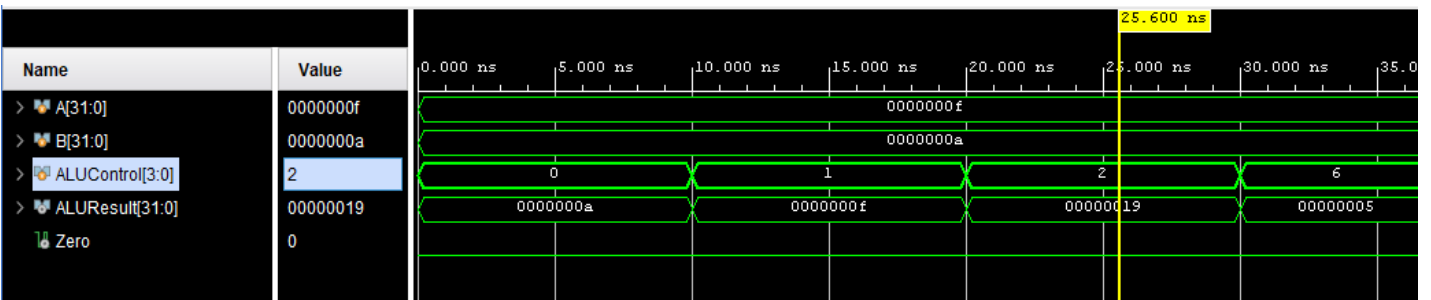
and



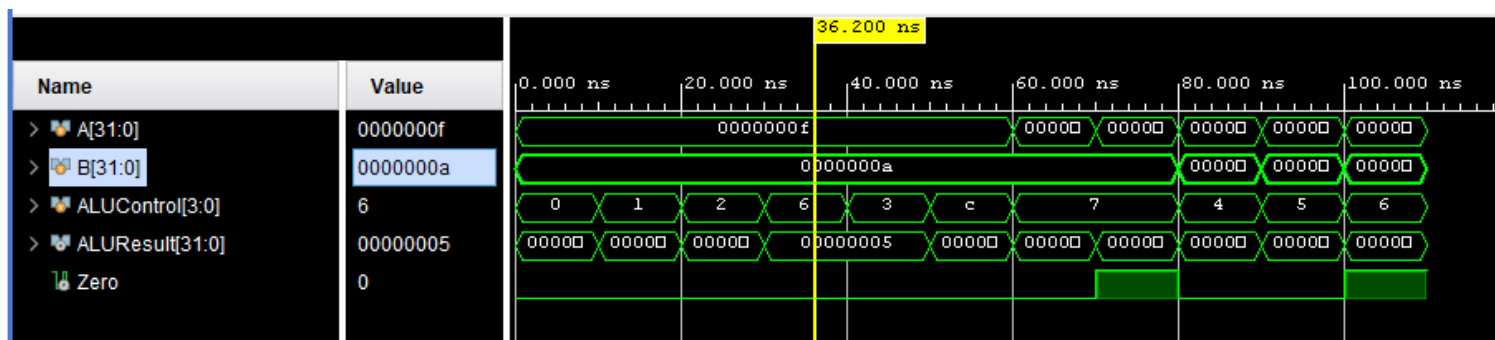
OR



ADD

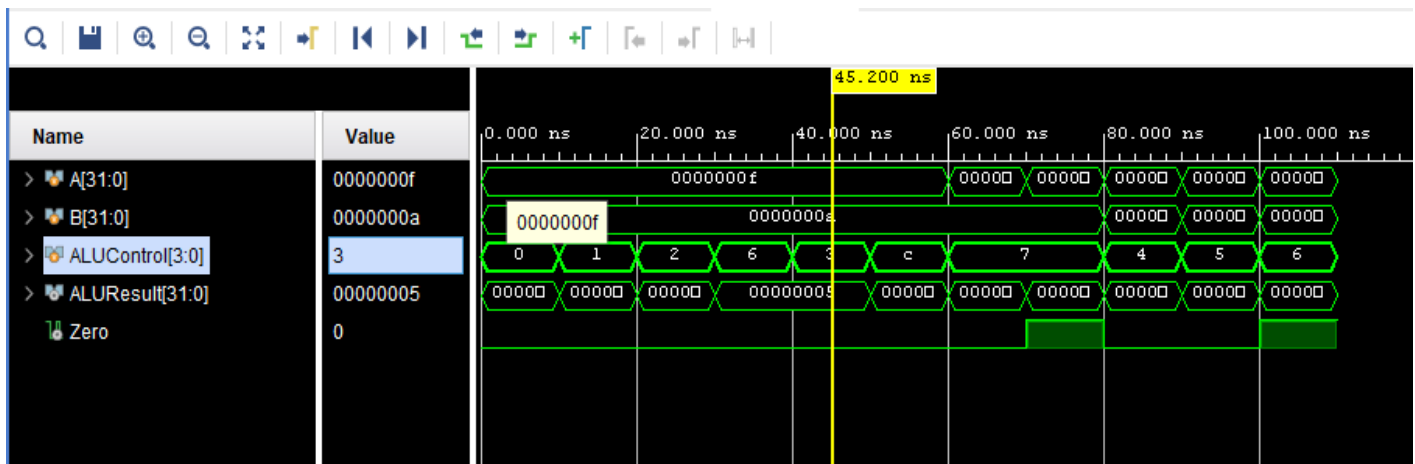


SUB

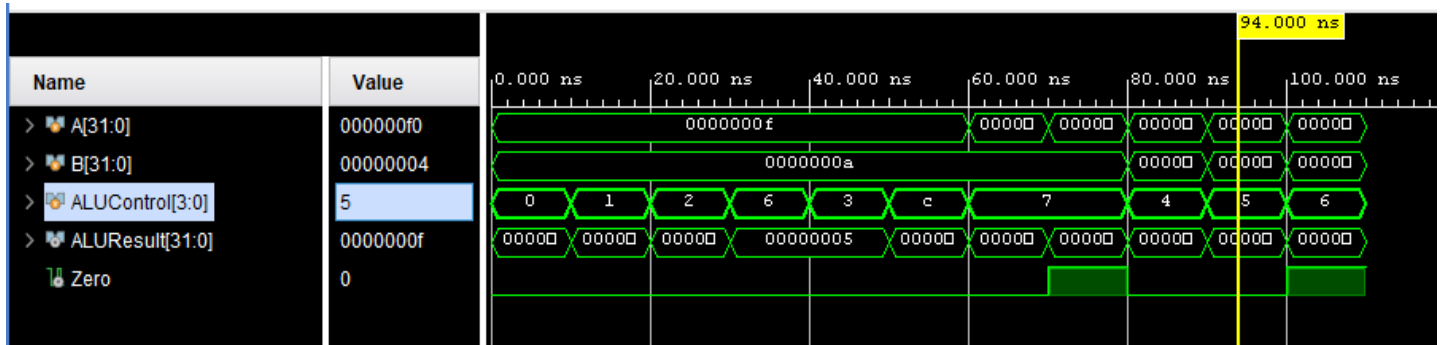




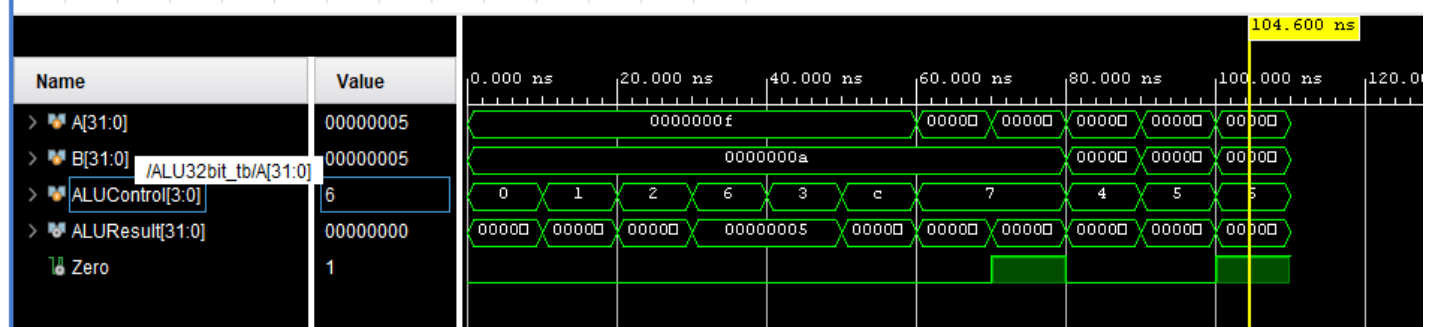
XOR



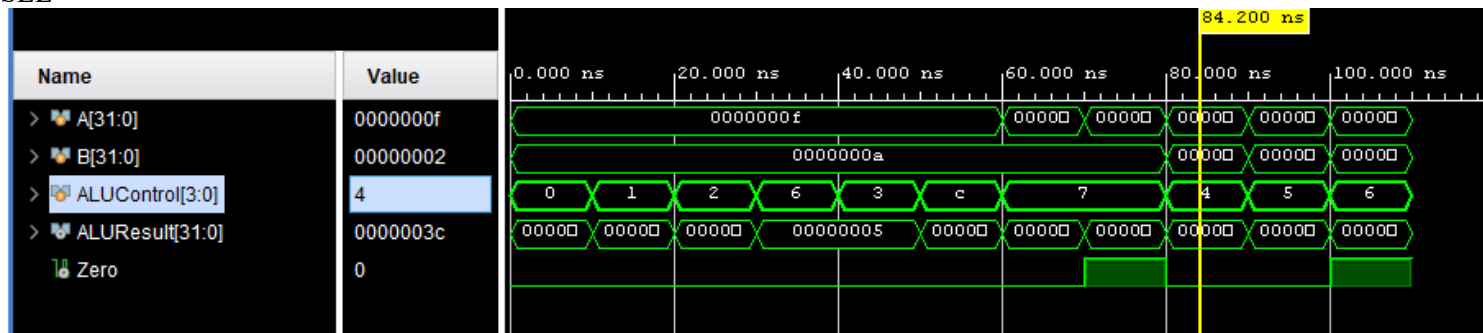
NOR



SLT

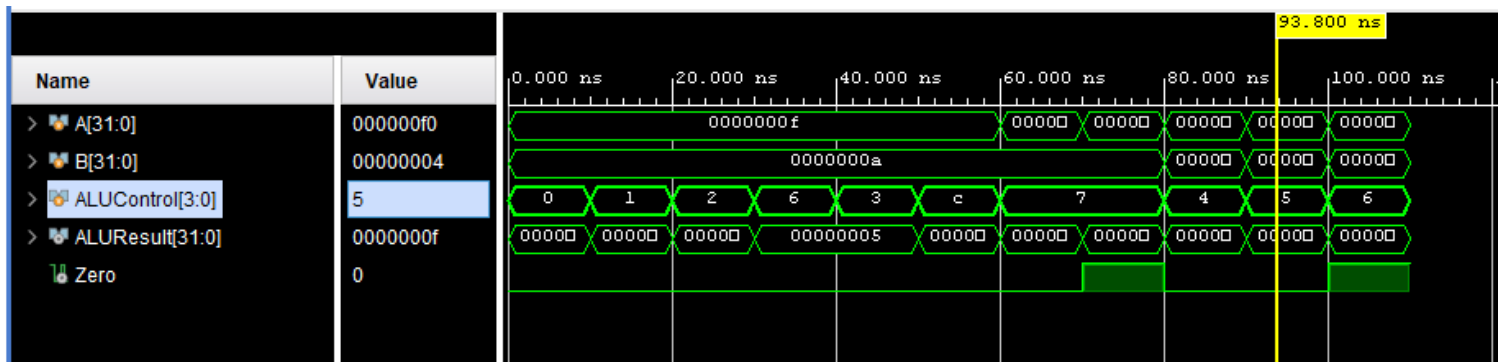


SLL





SRL

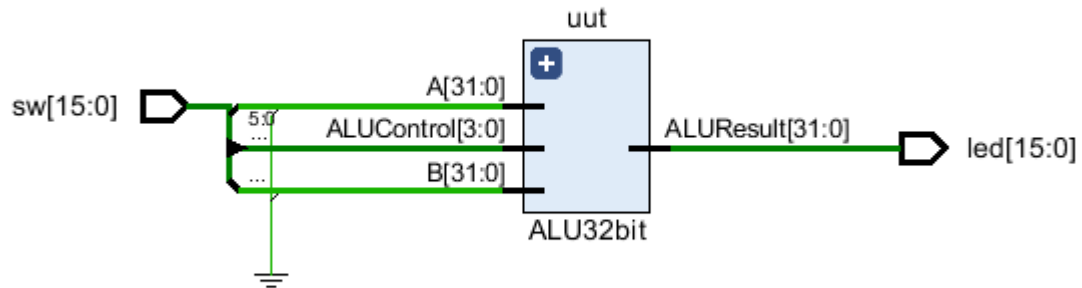


Truth table of test bench:

ALUControl	Operation	Expected Example (A=15, B=10)
0000	AND	15 & 10 = 10
0001	OR	15 10 = 15
0010	ADD	15 + 10 = 25
0110	SUB	15 - 10 = 5
0011	XOR	15 ^ 10 = 5
1100	NOR	~(15
0111	SLT	1 if A < B else 0
0100	SLL	15 << 2 = 60
0101	SRL	240 >> 4 = 15

Task 3: FPGA Synthesis and RTL Analysis

RTL schematic screenshots

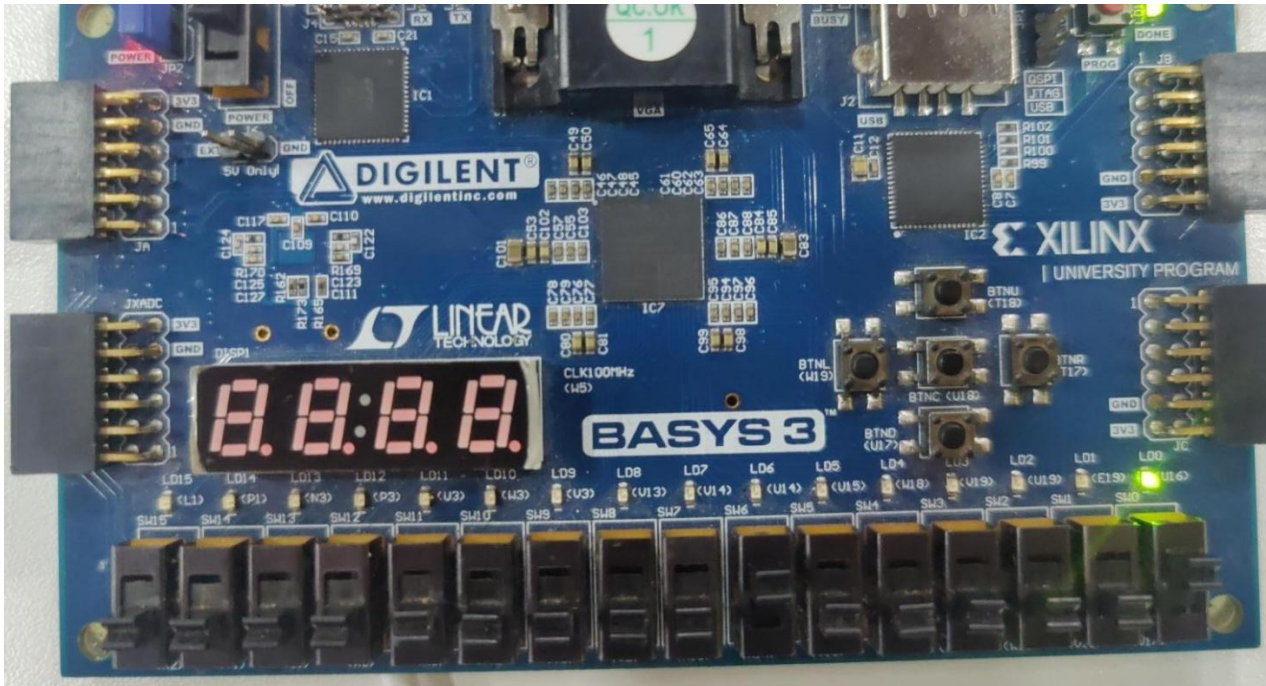


Demonstration of FPGA operation

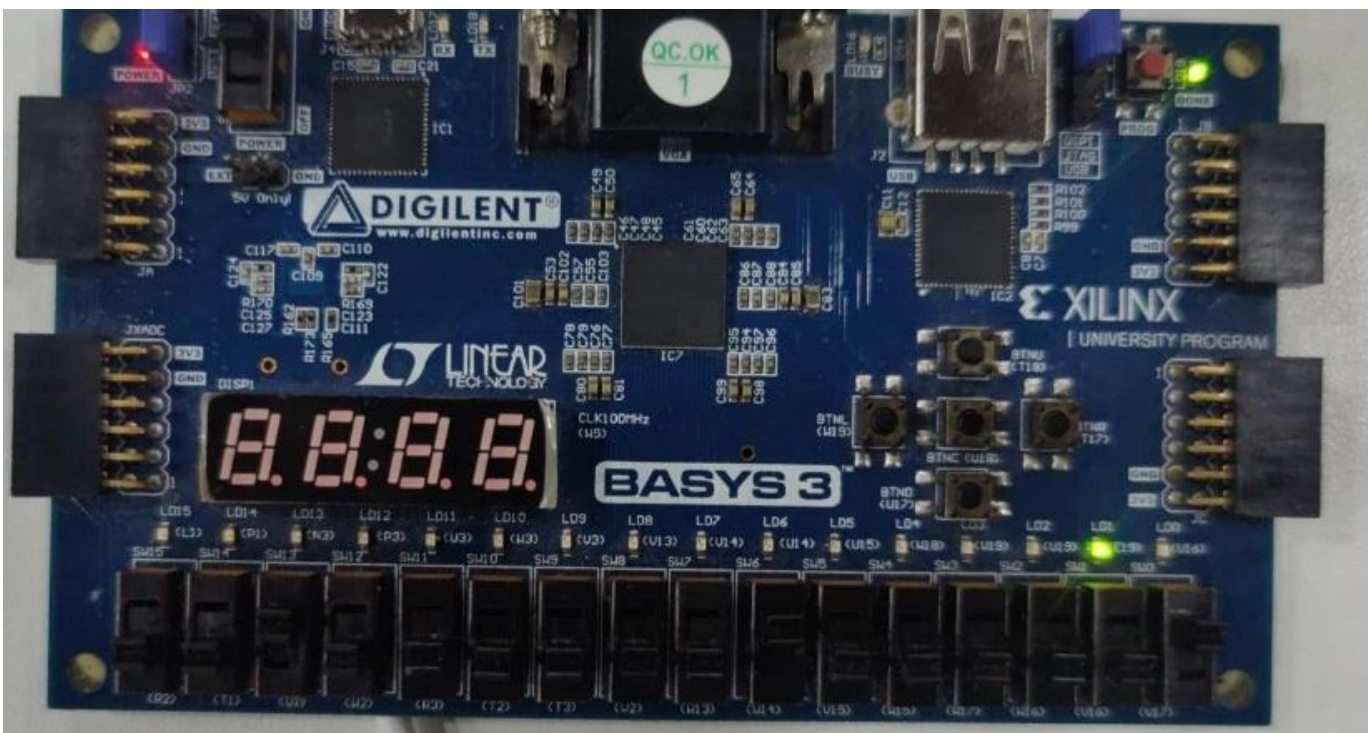
1. `sw[5:0]` → lower 6 bits of A.
2. `sw[11:6]` → lower 6 bits of B.
3. `sw[15:12]` → **ALUControl** to select operation.
4. `led[15:0]` will display the lower 16 bits of the ALU result.
5. `#10` gives 10 ns delay between test cases for simulation visibility.
6. `$finish` stops the simulation.

FPGA PICTURES:

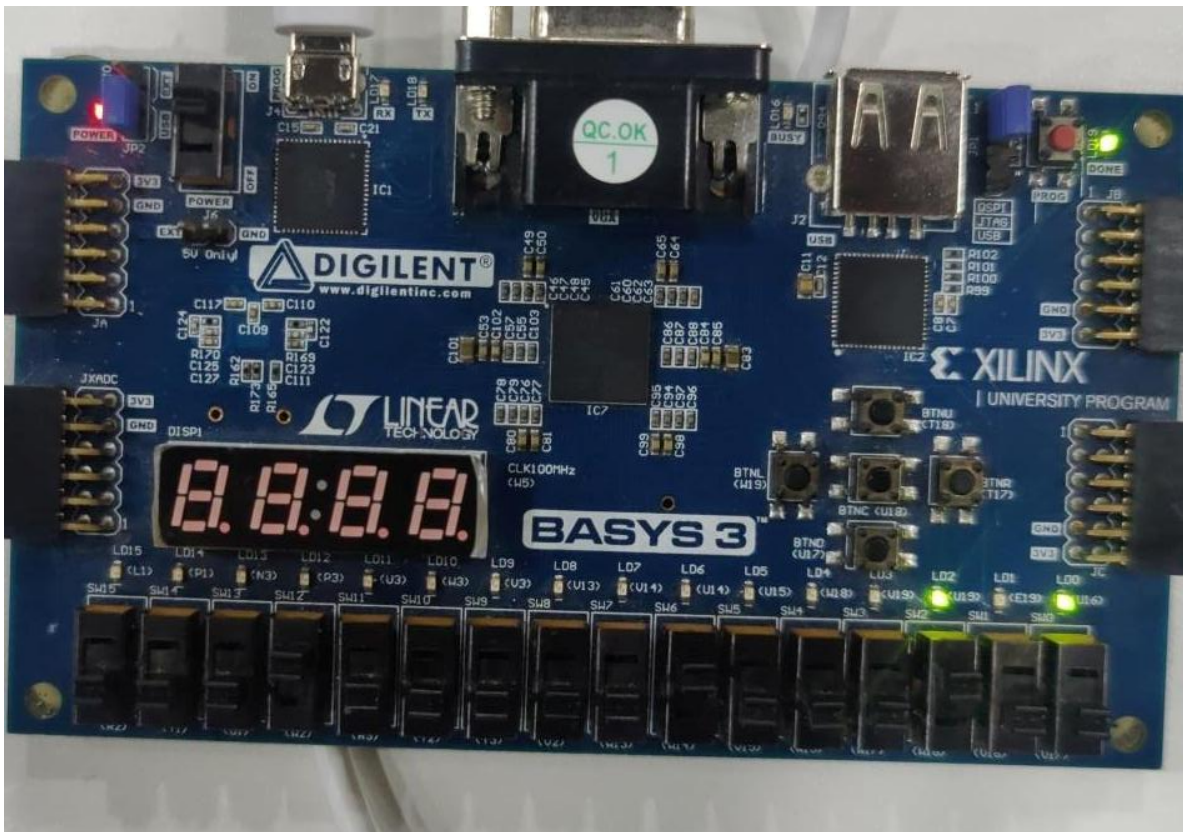
AND:



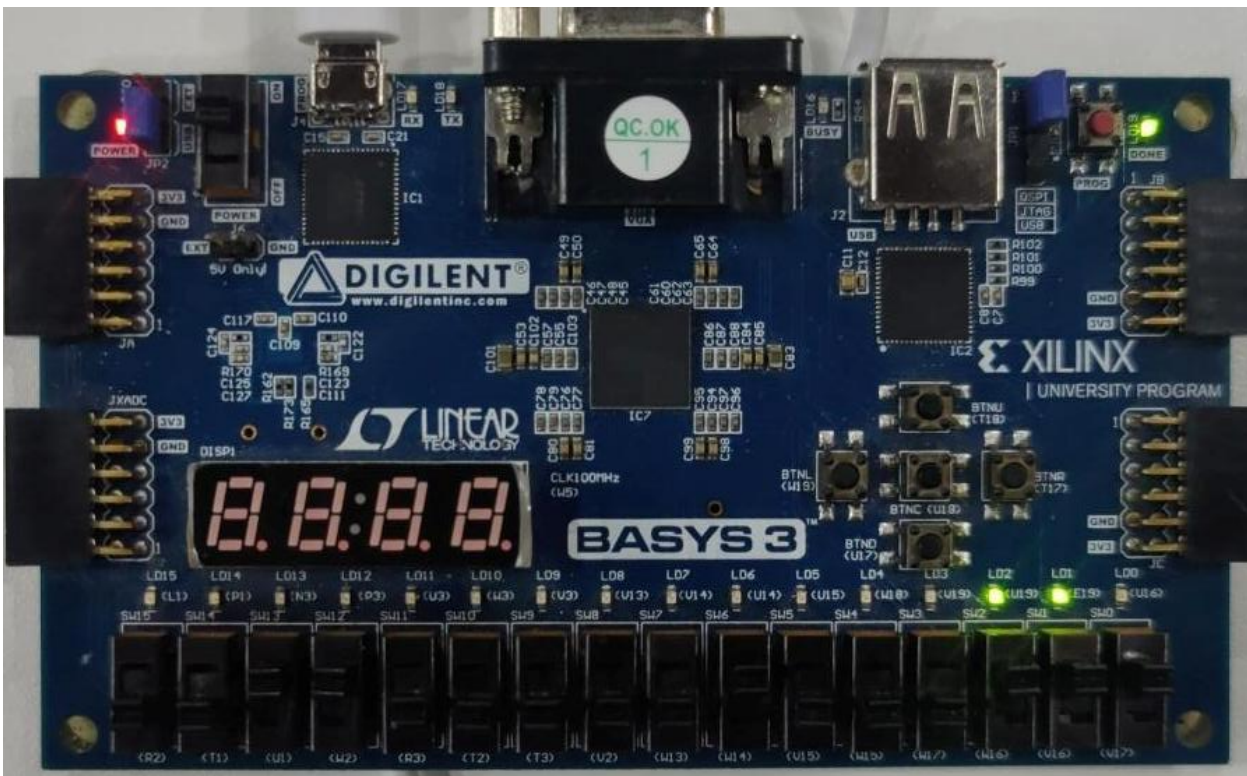
ADD:



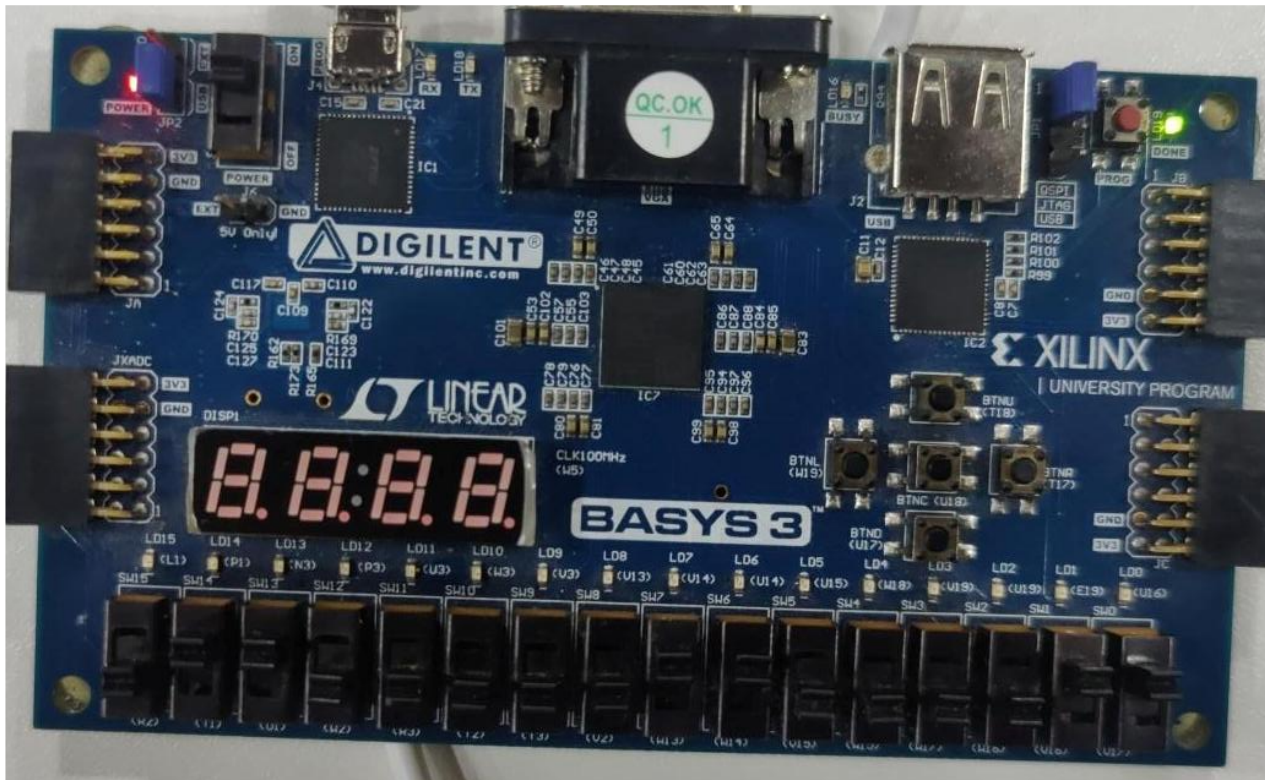
OR:



XOR:



SUB:





Assessment Rubric

Lab 6: Design and FPGA Implementation of the ALU for a Single-Cycle RISC-V Processor

Name:	Student ID:	Section:
--------------	--------------------	-----------------

Points Distribution

	Task No.	LR 2 (Code)	LR 5 (Results)	LR 11 (Design)
In-Lab	Task 1	-	-	/20
	Task 2	/20	/15	-
	Task 3	/10	/15	-
Total Points		/30	/30	/20
CLO Mapped		CLO 1		

Affective Domain Rubric		Points	CLO Mapped
AR7	Report Submission & Git Upload	/10 & /10	CLO 1

CLO	Total Points	Points Obtained
1	100	
Total	100	

For description of different levels of the mapped rubrics, please refer to the Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics provided here.